Experiment-4

Aim: Implement CPU scheduling algorithms (c) Round Robin (b) SJF (a) FCFS.

a)FCFS

Description: FCFS is a non-preemptive CPU scheduling algorithm where processes are executed completely in the order in which they arrive. The CPU always serves the earliest-arriving process first, and later processes wait until the current process finishes.

Algorithm:

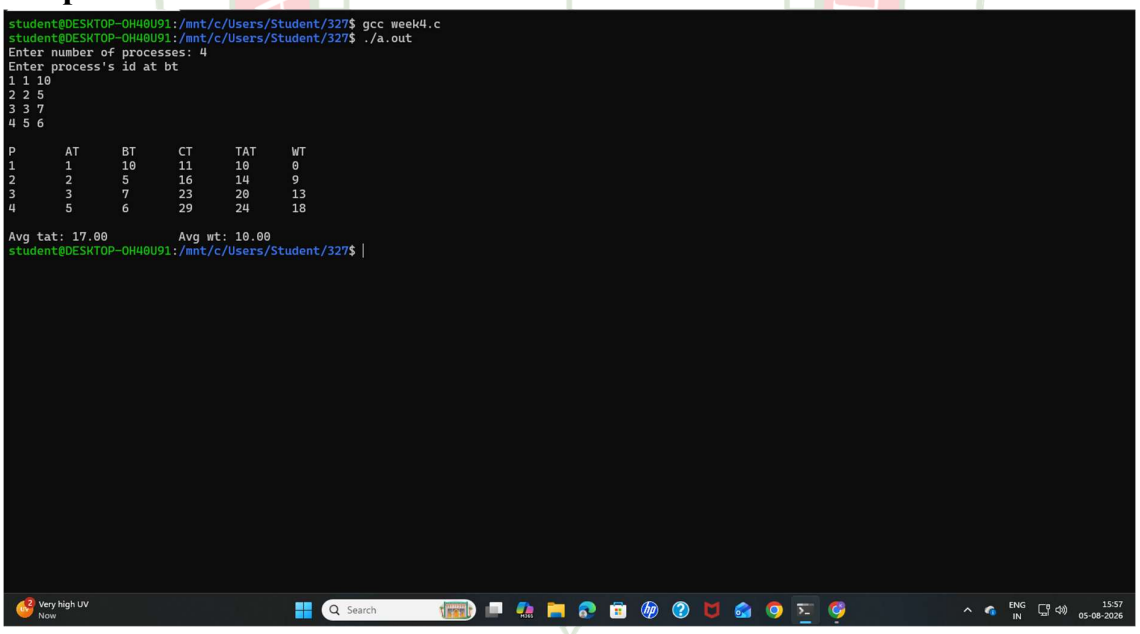
1. Read the number of processes.
2. Read PID, Arrival Time, and Burst Time for each process.
3. Sort processes in ascending order of Arrival Time.
4. Initialize CPU time t = 0.
5. For each process:
    1. If t < AT, set t = AT.
    2. Calculate CT = t + BT.
    3. Calculate TAT = CT - AT.
    4. Calculate WT = TAT - BT.
    5. Update total TAT and WT.
6. Display the scheduling table.
7. Calculate and display average TAT and average WT.

Code:

```
#include<stdio.h>
struct process {
int id;
int at;
int bt;
int ct;
int tat;
int wt;
};
void sort(struct process p[], int n){
for(int i=0; i<n-1;i++){
for(int j=0;j<n-i-1;j++){
if(p[j].at > p[j+1].at){
struct process temp;
temp = p[j+1];
p[j+1] = p[j];
p[j] = temp;
}
}
}
return;
}
int main(){
int n;
printf("Enter number of processes: ");
scanf("%d" , &n);
struct process p[n];
printf("Enter process's id at bt\n");
for(int i =0;i<n;i++){
scanf("%d %d %d" , &p[i].id,&p[i].at,&p[i].bt);
}
sort(p, n);
```

```
int t=0;
float ttat = 0;
float twt =0;
for(int i=0; i<n; i++){
if(t < p[i].at){
t = p[i].at;
}
t += p[i].bt;
p[i].ct = t;
p[i].tat = p[i].ct - p[i].at;
p[i].wt = p[i].tat - p[i].bt;
ttat += p[i].tat;
twt += p[i].wt;
}
printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
for(int i =0; i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t%d\t",
p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
printf("\n");
}
printf("\nAvg tat: %.2f\t\tAvg wt: %.2f\n", ttat/n,twt/n);
return 0;
}
```

Output:



b)SJF without Arrival Time

Description: SJF is a non-preemptive CPU scheduling algorithm that selects the process with the shortest burst time and executes it completely before selecting the next shortest process. The main objective of SJF is to minimize the average waiting time and average turnaround time.

Algorithm:

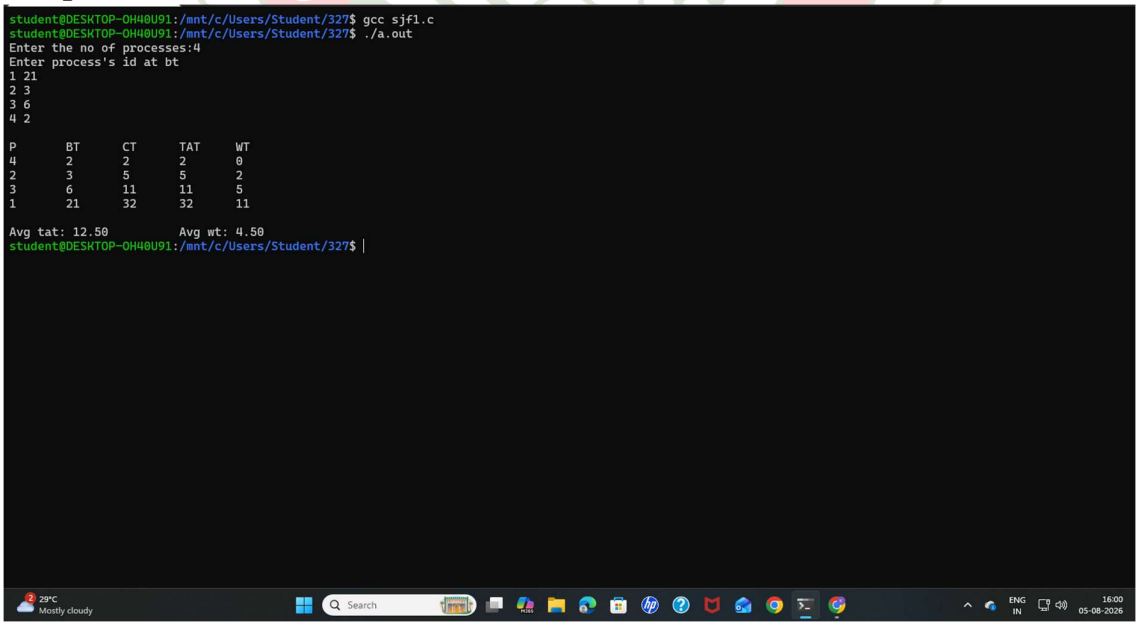
1. **Start**
2. Read the number of processes n.
3. For each process, read its **Process ID (PID)** and **Burst Time (BT)**.
4. Sort all processes in **ascending order of Burst Time**.
5. Initialize the current time t = 0.
6. For each process in the sorted order:
   - Calculate **Completion Time**:
     CT = t + BT
   - Update current time:
     t = CT
   - Calculate **Turnaround Time**:
     TAT = CT
     (Since all processes arrive at time 0.)
   - Calculate **Waiting Time**:
     WT = TAT - BT
7. Calculate the **total Turnaround Time** and **total Waiting Time**.
8. Calculate:
   - **Average TAT = Total TAT / n**
   - **Average WT = Total WT / n**
9. Display the **PID, BT, CT, TAT, WT**, Average TAT, and Average WT.
10. **Stop**.

Code:

```
#include<stdio.h>
struct process {
int id;
int bt;
int ct;
int tat;
int wt;
};
void sort(struct process p[], int n){
for(int i=0; i<n-1;i++){
for(int j=0;j<n-i-1;j++){
if(p[j].bt > p[j+1].bt){
struct process temp;
temp = p[j+1];
p[j+1] = p[j];
p[j] = temp;
}
}
}
return;
}
int main(){
int n;
```

```
printf("Enter the no of processes:");
scanf("%d" , &n);
struct process p[n];
printf("Enter process's id at bt\n");
for(int i =0;i<n;i++){
scanf("%d %d" , &p[i].id,&p[i].bt);
}
sort(p,n);
int t=0;
float ttat = 0;
float twt =0;
for(int i=0; i<n; i++){
t += p[i].bt;
p[i].ct = t;
p[i].tat = p[i].ct;
p[i].wt = p[i].tat - p[i].bt;
ttat += p[i].tat;
twt += p[i].wt;
}
printf("\nP\tBT\tCT\tTAT\tWT\n");
for(int i =0; i<n;i++){
printf("%d\t%d\t%d\t%d\t%d\t", p[i].id,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
printf("\n");
}
printf("\nAvg tat: %.2f\t\tAvg wt: %.2f\n", ttat/n,twt/n);
return 0;
}
```

Output:



b) 1)SJF with Arrival Time

Description: SJF is a non-preemptive CPU scheduling algorithm that selects the process with the shortest burst time and executes it completely before selecting the next shortest process. The main objective of SJF is to minimize the average waiting time and average turnaround time.

Algorithm:

1. Start
2. Read the number of processes n.
3. For each process:
   - Assign a Process ID.
   - Read its Arrival Time (AT) and Burst Time (BT).
   - Mark the process as not completed.
4. Initialize:
   - current_time = 0
   - completed_count = 0
   - total_TAT = 0
   - total_WT = 0
5. Repeat until all processes are completed:
   - Find all uncompleted processes whose Arrival Time ≤ current time.
   - Among them, select the process with the smallest Burst Time.
   - If two processes have the same Burst Time, select the one with the earlier Arrival Time.
6. If a suitable process is found:
   - Calculate Completion Time:
     CT = current_time + BT
   - Calculate Turnaround Time:
     TAT = CT - AT
   - Calculate Waiting Time:
     WT = TAT - BT
   - Add TAT and WT to their respective totals.
   - Mark the process as completed.
   - Increment completed_count.
   - Update:
     current_time = CT
7. If no process has arrived yet, increment current_time by 1.
8. After all processes are completed, calculate:
   - Average TAT = Total TAT / n
   - Average WT = Total WT / n
9. Display the PID, Arrival Time, Burst Time, Completion Time, Turnaround Time, and Waiting Time.
10. Display the Average Turnaround Time and Average Waiting Time.
11. Stop.

Code:

```
#include <stdio.h>
#include <stdbool.h>
struct Process {
    int pid;
```

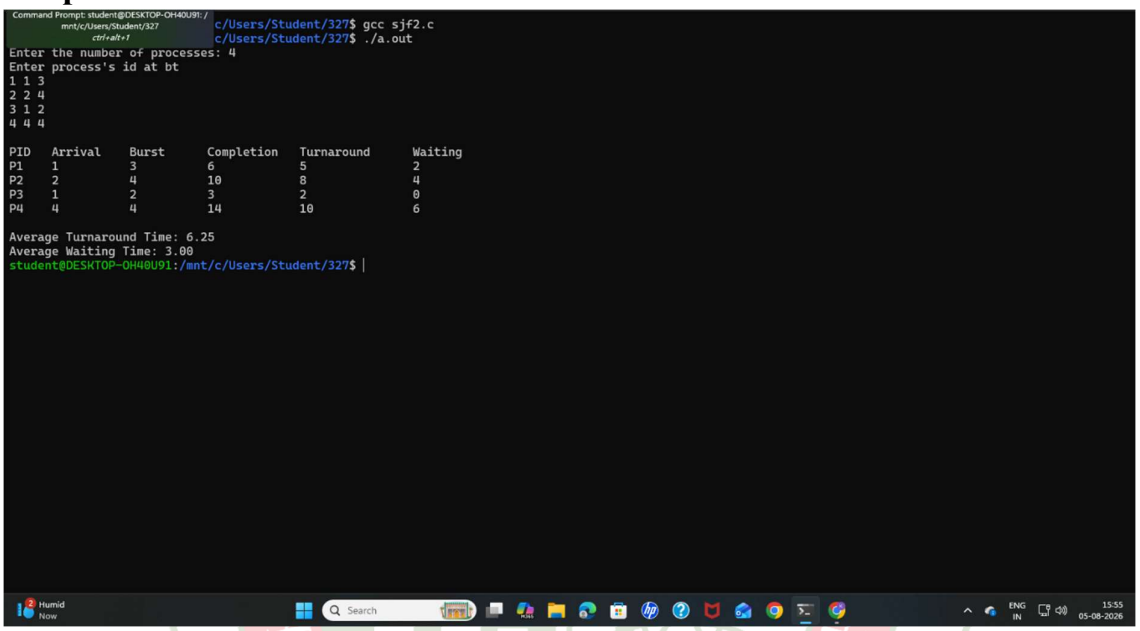
```
    int arrival;
    int burst;
    int completion;
    int turnaround;
    int waiting;
    bool completed;
};
int main() {
    int n;
    printf("Enter the number of processes: ");
    if (scanf("%d", &n) != 1 || n <= 0) return 1;

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter Arrival Time and Burst Time for Process P%d: ",
p[i].pid);
        scanf("%d %d", &p[i].arrival, &p[i].burst);
        p[i].completed = false;
    }
    int current_time = 0;
    int completed_count = 0;
    float total_tat = 0, total_wt = 0;
    while (completed_count < n) {
        int idx = -1;
        int min_burst = 1e9;
        for (int i = 0; i < n; i++) {
            if (p[i].arrival <= current_time && !p[i].completed) {
                if (p[i].burst < min_burst) {
                    min_burst = p[i].burst;
                    idx = i;
                } else if (p[i].burst == min_burst) {
                    if (p[i].arrival < p[idx].arrival) {
                        idx = i;
                    }
                }
            }
        }
        if (idx != -1) {
            p[idx].completion = current_time + p[idx].burst;
            p[idx].turnaround = p[idx].completion - p[idx].arrival;
            p[idx].waiting = p[idx].turnaround - p[idx].burst;
            total_tat += p[idx].turnaround;
            total_wt += p[idx].waiting;
            p[idx].completed = true;
            completed_count++;
            current_time = p[idx].completion;
        } else {
            current_time++;
        }
    }
    printf("\n%-5s %-10s %-10s %-12s %-15s %-12s\n",
           "PID", "Arrival", "Burst", "Completion", "Turnaround", "Waiting");
    for (int i = 0; i < n; i++) {
        printf("P%-4d %-10d %-10d %-12d %-15d %-12d\n",
               p[i].pid, p[i].arrival, p[i].burst,
```

```
                p[i].completion, p[i].turnaround, p[i].waiting);
    }
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);
    return 0;
}
```

Output:



INSTITUTE OF TECHNOLOGY
స్వయం తేజస్విన్ భవ
1979